



GESTION DES ÉVÉNEMENTS

Introduction

Les événements de Laravel fournissent une implémentation simple du modèle d'observateur, vous permettant de souscrire et d'écouter divers événements qui se produisent dans votre application. Les classes d'événements sont généralement stockées dans le répertoire **app/Events**, tandis que leurs écouteurs sont stockés dans **app/Listeners**. Ne vous inquiétez pas si vous ne voyez pas ces répertoires dans votre application car ils seront créés pour vous lorsque vous générerez des événements et des écouteurs à l'aide des commandes de la console Artisan.

ENREGISTREMENT DES ÉVÉNEMENTS ET DES ÉCOUTEURS



Introduction

L'`App\Providers\EventServiceProvider` inclus dans votre application Laravel fournit un endroit pratique pour enregistrer tous les écouteurs d'événements de votre application. La propriété **`listen`** contient un tableau de tous les événements (clés) et de leurs écouteurs (valeurs). Vous pouvez ajouter autant d'événements à ce tableau que votre application le nécessite.

Par exemple, ajoutons un événement `OrderShipped` :

```
protected $listen = [  
    OrderShipped::class => [  
        SendShipmentNotification::class,  
    ],  
];
```

Génération d'événements/listeners

vous pouvez utiliser les commandes artisanales **make:event** et **make:listener** pour générer des événements et des listeners individuels :

```
php artisan make:event PodcastProcessed
```

```
php artisan make:listener SendPodcastNotification --event=PodcastProcessed
```

Vous pouvez aussi ajouter des listeners et des événements à votre **EventServiceProvider** et utiliser la commande **event:generate**. Cette commande générera tous les événements ou listeners qui sont listés dans votre **EventServiceProvider** et qui n'existent pas encore :

```
php artisan event:generate
```

DÉFINIR LES ÉVÉNEMENTS



Définir les événements

Une classe d'événement est essentiellement un conteneur de données qui contient les informations liées à l'événement.

Par exemple, supposons qu'un événement `App\Events\OrderShipped` reçoit un objet ORM Eloquent, cette classe d'événement ne contient aucune logique. C'est un conteneur pour l'instance de commande Eloquent qui a été achetée. Le trait `SerializesModels` utilisé par l'événement sérialise tout modèle Eloquent si l'objet de l'événement est sérialisé à l'aide de la fonction `serialize` de PHP.

```
class OrderShipped
{
    use Dispatchable, InteractsWithSockets, SerializesModels;

    /**
     * The order instance.
     *
     * @var \App\Models\Order
     */
    public $order;

    /**
     * Create a new event instance.
     *
     * @param \App\Models\Order $order
     * @return void
     */
    public function __construct(Order $order)
    {
        $this->order = $order;
    }
}
```

DÉFINIR LES ÉCOUTEURS (LISTNERS)

Définir les écouteurs (listeners)

Les écouteurs d'événements reçoivent des instances d'événements dans leur méthode **handle**. Les commandes Artisan **event:generate** et **make:listener** importeront automatiquement la classe d'événement appropriée et indiqueront le type d'événement dans la méthode **handle**. Dans la méthode **handle**, vous pouvez effectuer toutes les actions nécessaires pour répondre à l'événement :

```
<?php
```

```
namespace App\Listeners;
```

```
use App\Events\OrderShipped;
```

```
class SendShipmentNotification
```

```
{
```

```
    public function __construct()
```

```
    {
```

```
        //
```

```
    }
```

```
    public function handle(OrderShipped $event)
```

```
    {
```

```
        // Access the order using $event->order...
```

```
    }
```

```
}
```

• + ENVOI D'ÉVÉNEMENTS • + ◯

Distribuer un événement

Pour distribuer un événement, vous pouvez appeler la méthode **dispatch** sur l'événement. Cette méthode est rendue disponible sur l'événement par **Illuminate\Foundation\Events\Dispatchable**. Tous les arguments transmis à la méthode de répartition seront transmis au constructeur de l'événement :

```
public function store(Request $request)
{
    $order = Order::findOrFail($request->order_id);

    //...

    OrderShipped::dispatch($order);
}
```